

Thiết kế Server AI Phân loại & Định tuyến Văn bản Đến

1. Bài toán

Xây dựng một server AI cho văn phòng Sở. **Đầu vào** là một văn bản đến cùng **file đính kèm (thường là PDF)** — **không còn payload JSON** do người dùng nhập tay; toàn bộ thông tin cần thiết được **trích xuất tự động từ nội dung file**.

Đầu ra — kết quả để **chuyển văn bản đi**, gồm 4 trường:

Trường	Ý nghĩa
Đơn vị xử lý chính	Người/đơn vị chịu trách nhiệm xử lý chính (có thể kèm lãnh đạo phụ trách)
Phối hợp xử lý	Người/đơn vị phối hợp xử lý
Lãnh đạo theo dõi	Lãnh đạo theo dõi, chỉ đạo
Hạn thực hiện	Thời hạn cần hoàn thành xử lý (trích từ nội dung văn bản, hoặc null nếu không xác định)

Kết quả được suy ra từ: **trích nội dung PDF** → **trích metadata** (số hiệu, loại, cơ quan ban hành, người ký, ngày, trích yếu, hạn) → **áp dụng Rule** của Sở tương ứng.

Knowledge base được xây dựng **riêng cho từng Sở** (mỗi Sở 1 bộ `rulebase` + bảng nhân sự riêng). Trước mắt triển khai cho **Sở NN&MT** (Gia Lai); kiến trúc để mở rộng sang Sở khác bằng cách thêm bộ rule tương ứng.

Ràng buộc kỹ thuật:

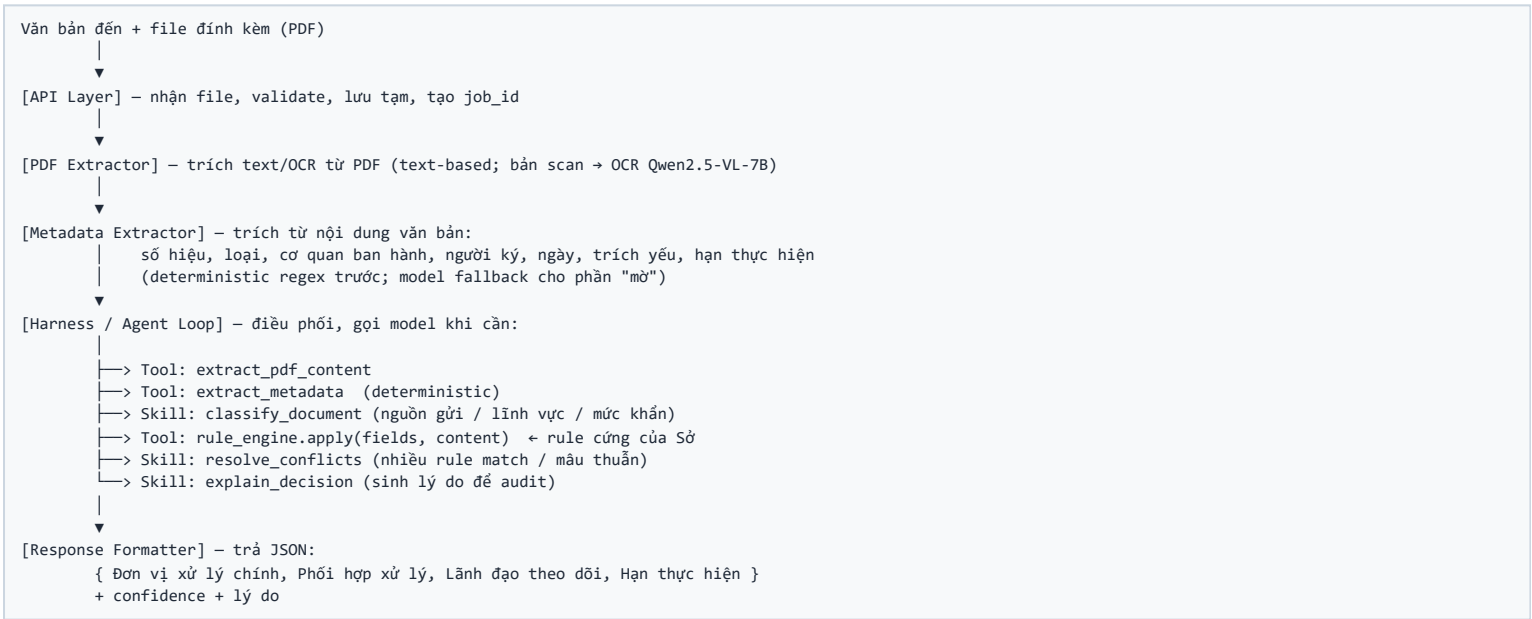
- Kiến trúc theo mô hình **Orchestration, Harness, Agent Skills**; backend là **open-weight model tự host tại công ty** (offline, không gọi API model bên ngoài).
- Không sử dụng RAG** — rule quản lý dạng rule engine tường minh, không phải retrieval ngữ nghĩa.

2. Vì sao không dùng RAG

Rule do admin set là tập hữu hạn, có cấu trúc (điều kiện → cơ quan nhận). Nên xử lý bằng:

- Rule engine thuần code** (deterministic) cho điều kiện rõ ràng — nhanh, rẻ, dễ audit, dự đoán được.
- Model reasoning** (open-weight tự host) cho phần "mờ": trích metadata không rõ pattern, phân loại ngữ nghĩa, suy luận khi rule không match.
- Toàn bộ rule set của Sở được **load thẳng vào context** khi cần reasoning (không retrieval/embedding).
- Với khối lượng dự kiến ~600 văn bản/ngày, tận dụng **prefix caching** (vLLM/TGI) cho phần system prompt/rule set cố định.

3. Kiến trúc tổng thể



Vai trò từng thành phần

Thành phần	Vai trò
PDF Extractor	Trích text từ PDF (pdfplumber/pypdf); bản scan → OCR Qwen2.5-VL-7B (vLLM) + Tesseract fallback
Metadata Extractor	Trích các trường nghiệp vụ + hạn thực hiện từ văn bản; deterministic trước, model sau
Harness	Vòng lặp điều phối, retry, giới hạn bước, tự hạ cấp fallback T2 → T1 → T0
Rule Engine (code)	Áp dụng rule tường minh của Sở; tách khỏi model để nhanh/audit/dự đoán được
Agent Skills	Đóng gói từng nghiệp vụ: extract-metadata, classify-document, apply-routing-rules, resolve-conflicts, explain-decision
Inference Server (vLLM/TGI)	Chạy open-weight model tại chỗ, gọi qua API local, không cần Internet

Mô hình 3 bậc của Harness (fallback)

Harness hạ cấp dần mà kết quả vẫn hợp lệ — rule engine luôn chạy độc lập, model chỉ làm phần "mờ".

Bậc	Cách gọi model	Dùng khi
T2 — Tool-calling	Model tự gọi tool (extract_metadata, apply_rules...) qua vòng lặp	Model hỗ trợ function-calling tốt
T1 — Prompt-only	Gộp toàn bộ văn bản + rule set vào 1 prompt, model trả 1 JSON	Model không hỗ trợ tool-calling
T0 — Không model	Chỉ chạy extraction + rule engine deterministic; phần không rõ → flag người duyệt	Model lỗi/timeout/parse hỏng

Thang fallback: T2 → T1 → T0. Kết quả luôn kèm cờ `degraded` để audit.

Kết nối mạng & bảo mật dữ liệu

Văn bản có thể thuộc loại nhạy cảm/mật — **không rời khỏi mạng nội bộ**: tự host open-weight model trên GPU on-prem (vLLM/TGI), chạy hoàn toàn offline cho bước reasoning. Toàn bộ Harness, Rule Engine, inference server nằm cùng máy/cụm máy tại công ty.

4. Mô hình dữ liệu (input / output)

Input

POST /classify nhận **duy nhất file đính kèm** (multipart file), thường là PDF. Không có payload JSON. (Có trường text tùy chọn để test/dev khi chưa có file.)

Output (response)

```
{
  "job_id": "...",
  "don_vi_xu_ly_chinh": ["Chi cục Thủy sản"],
  "phoi_hop_xu_ly": ["Giám đốc Cao Thanh Thương"],
  "lanh_dao_theo_doi": ["Giám đốc Cao Thanh Thương"],
  "han_thuc_hien": "2026-08-27",
  "confidence": 0.95,
  "reason": "Khớp rule V.9 – thủy lợi (bình thường)",
  "matched_rules": ["V.9"],
  "needs_review": false,
  "degraded": false,
  "tier": "T0",
  "extracted_metadata": { "so_hieu": "...", "loai": "...", "co_quan_ban_hanh": "...", "ngay_van_ban": "...", "trich_yeu": "...", "han_thuc_hien": "..." }
}
```

Ánh xạ rule → 4 trường: xử lý chính → don_vi_xu_ly_chinh; phối hợp xử lý → phoi_hop_xu_ly; theo dõi → lanh_dao_theo_doi; hạn → han_thuc_hien.

Hạn thực hiện được trích từ nội dung văn bản theo thứ tự ưu tiên:

- 1. Ngày cụ thể: trước ngày ..., hạn ... ngày DD/MM/YYYY, chậm nhất ngày
- 2. Dấu hiệu khẩn: khẩn, hỏa tốc, thương khẩn → "hỏa tốc".
- 3. Không xác định được → null (cờ nhẹ để người duyệt tự xác định).

5. Explainability & Human review

- Mỗi kết quả có **confidence** (rule match rõ ràng vs model suy luận).
- Confidence thấp / nhiều rule mâu thuẫn / metadata không rõ → **flag needs_review** để người xác nhận.
- Log đầy đủ: rule nào match, model giải thích gì, metadata đã trích — phục vụ audit và tinh chỉnh rule.

6. Cấu trúc project

```
ai-doc-router/
├── app/
│   ├── main.py                # FastAPI entrypoint
│   ├── api/routes.py          # POST /classify – nhận file đính kèm
│   ├── pdf/extractor.py       # text extraction + OCR Qwen2.5-VL + Tesseract fallback
│   ├── extract/metadata.py    # trích metadata + hạn thực hiện từ văn bản
│   ├── rules/
│   │   ├── rules.yaml        # rulebase riêng từng Sở (SoNNMT)
│   │   └── engine.py         # rule matcher deterministic
│   ├── orchestrator/
│   │   ├── harness.py        # vòng lặp agent + fallback T2→T1→T0
│   │   └── skills/           # SKILL.md cho từng bước reasoning
│   ├── models/schema.py       # Pydantic: response 4 trường
│   └── storage/audit.py       # SQLite log kết quả + lý do
├── tests/
├── tools/                     # build_pdf.py (sinh PDF rulebase/kế hoạch)
├── requirements.txt
└── .env.example
```

7. Tech stack

- API**: FastAPI (Python)
- PDF extraction**: pdfpplumber / pypdf (text-based); bản scan → **Qwen2.5-VL-7B-Instruct qua vLLM** (chính) + Tesseract (fallback)
- Metadata extraction**: regex thuần (deterministic) + model fallback
- Rule storage**: YAML/JSON (rulebase riêng từng Sở), sau chuyển SQLite/Postgres + admin UI
- Orchestration**: harness tự viết, gọi model qua API tương thích OpenAI do vLLM/TGI cung cấp
- Model định tuyến**: self-host open-weight (Qwen2.5/Qwen3, Llama... — benchmark lại tại thời điểm triển khai)
- Model OCR**: Qwen2.5-VL-7B-Instruct (vLLM) cho bản scan
- Phần cứng**: NVIDIA RTX 4090 (24 GB VRAM) — serve OCR VLM + model định tuyến qua vLLM on-prem
- Audit**: SQLite (demo) → Postgres (production)

8. Lộ trình thực hiện

Giai đoạn 1 — Demo local (Sở NN&MT)

- 0. **Thu thập 20–30 văn bản mẫu thật + xác minh OCR** — kiểm tra tỷ lệ scan vs text gốc TRƯỚC khi code; chốt OCR dùng Qwen2.5-VL-7B (vLLM) + Tesseract fallback.
- 1. **PDF extractor** — trích text từ PDF (có sẵn).
- 2. **Metadata extractor** — trích số hiệu, loại, cơ quan, ngày, trích yếu, hạn (deterministic).
- 3. **Rule engine** — rules.yaml SoNNMT + matcher deterministic, output 4 trường (rule theo chức danh, xem Mục 11.2).

4. **Harness + model local** — gọi model cho phần metadata/classification "mở"; fallback T2→T1→T0. **Benchmark model sơ bộ SONG SONG** để chốt khả năng tool-calling (bậc T2) trước khi cam kết kiến trúc.
5. **Agent Skills** — hướng dẫn từng bước reasoning.
6. **Test end-to-end** + đánh giá theo tiêu chí định lượng (Mục 10).
7. **Audit log** — lưu input, metadata trích, rule matched, output vào SQLite.

Giai đoạn 2 — Triển khai server GPU tại công ty, 24/24

8. Docker + docker-compose, secret qua `.env`.
9. Reverse proxy (Nginx) trong mạng nội bộ, không expose Internet.
10. Giám sát uptime, backup rule, restart tự động.
11. Cài vLLM trên RTX 4090: serve **Qwen2.5-VL-7B-Instruct** (OCR) + model định tuyến đã benchmark; Harness gọi qua localhost/LAN.

Giai đoạn 3 — Mở rộng đa Sở

12. Tách rulebase thành cấu trúc "một Sở = một bộ rule + danh bạ", thêm endpoint/param chọn Sở.
13. Admin UI quản lý rule; chuẩn hóa danh bạ nhân sự giữa các Sở.

9. Các điểm cần xác nhận / làm rõ

- ☒ **Đầu vào:** chỉ văn bản + file đính kèm (PDF), không payload JSON.
- ☒ **Đầu ra:** 4 trường {Đơn vị xử lý chính, Phối hợp xử lý, Lãnh đạo theo dõi, Hạn thực hiện}.
- ☒ **Knowledge base:** rulebase riêng từng Sở; trước mắt Sở NN&MT.
- ☒ **Hạn thực hiện:** trích từ nội dung văn bản; văn bản loại khẩn → "hỏa tốc"; không xác định được → null.
- ☒ **Định dạng output:** khóa snake_case (don_vi_xu_ly_chinh...) cho client nhận.
- ☐ **OCR bản scan:** cần xác minh SƠM bằng 20–30 văn bản thật (scan phổ biến trong văn bản hành chính) — xem Mục 8 bước 0; OCR dùng Qwen2.5-VL-7B (vLLM) + Tesseract fallback.
- ☐ **Rule theo chức danh thay vì tên người:** chốt thiết kế map chức danh → người đang giữ chức qua bảng nhân sự riêng (Mục 11.2).
- ☐ **Trách nhiệm pháp lý khi định tuyến sai:** quy trình sign-off + người chịu trách nhiệm cuối (đưa ra bản với lãnh đạo).

10. Tiêu chí thành công định lượng (Giai đoạn 1)

Chỉ số	Ngưỡng đề xuất
Độ chính xác trích metadata	≥ 95% trên 30 văn bản mẫu
Độ chính xác định tuyến	≥ 90% trên 30 văn bản mẫu
Tỷ lệ needs_review (case "mở" cần người duyệt)	≤ 20%
Thời gian xử lý / văn bản	≤ 3 giây (chưa tính OCR)

Các ngưỡng trên là đề xuất ban đầu, cần lãnh đạo chốt trước khi xem là "đạt" để lên Giai đoạn 2.

11. Rủi ro & quyết định thiết kế (phản hồi đánh giá)

1. **OCR không để cuối** — văn bản hành chính thường scan; đưa bước thu thập mẫu + xác minh OCR lên **bước 0** (Mục 8). Dùng **Qwen2.5-VL-7B (vLLM)** cho bản scan, Tesseract làm fallback.
2. **Rule theo chức danh, không theo tên** — rule tham chiếu chức danh/vai trò (VD: `PGĐ Thủy Lợi`, `Giám đốc Sở`); một **bảng nhân sự riêng** map chức danh → người đang giữ chức (kèm email), cập nhật độc lập khỏi rule logic.
3. **Tiêu chí thành công định lượng** — đã thêm ở Mục 10.
4. **Trách nhiệm pháp lý khi AI sai** — kỹ thuật chỉ dừng ở cờ `needs_review`; cần quy trình sign-off do lãnh đạo quyết định.
5. **Prompt injection từ nội dung văn bản** — rule engine là code deterministic (không thực thi lệnh từ text); model chỉ trả JSON có cấu trúc; thêm bước **validate/sanitize output** trước khi dùng.
6. **Benchmark model song song** — chốt khả năng tool-calling (bậc T2) sớm cùng lúc xây khung, tránh sửa kiến trúc muộn.

12. Bước tiếp theo đề xuất

- Thu thập 20–30 văn bản mẫu thật + xác minh OCR (bước 0).
- Refactor rule sang chức danh + bảng nhân sự riêng (Mục 11.2).
- Thêm bước validate/sanitize output model (Mục 11.5).
- Benchmark model sơ bộ song song (Mục 11.6).